

Kamień Milowy II – Implementacja systemu

Aplikacja do zarządzania zadaniami

Bartłomiej Karkoszka

Rafał Grot

Filip Stępień

Przedmiot: Modelowanie i Analiza Systemów Informatycznych

Grupa: 1ID21A

Spis treści

1	Wprowadzenie	2
2	Stan implementacji	2
2.1	Struktura repozytorium	2
2.2	Funkcjonalności zaimplementowane	2
2.3	Lista endpointów REST	3
2.4	Schemat bazy danych	3
3	Stos technologiczny – aktualizacja	3
4	Zaktualizowane diagramy z kamienia I	4
4.1	Diagram przypadków użycia	4
4.2	Diagram klas	5
4.3	Diagram komponentów	5
4.4	Diagram warstw systemu	7
5	Diagram sekwencji – utworzenie zadania	7
6	Diagram aktywności – zarządzanie zadaniem	9
7	Uruchamianie i weryfikacja	9
7.1	Wymagania wstępne	9
7.2	Kroki uruchomienia	10
7.3	Weryfikacja	10
8	Podsumowanie	10

1 Wprowadzenie

Niniejszy dokument stanowi raport z drugiego kamienia milowego projektu *Aplikacja do zarządzania zadaniami*. Etap ten obejmuje:

- implementację podstawowych funkcjonalności systemu (REST API, persystencja, autoryzacja, interfejs użytkownika),
- przygotowanie diagramu sekwencji dla wybranego scenariusza,
- przygotowanie diagramu aktywności dla wybranego procesu systemowego,
- aktualizację wcześniej wykonanych diagramów (przypadków użycia, klas, komponentów oraz warstw) zgodnie z rzeczywistą implementacją.

W stosunku do pierwszego kamienia milowego nastąpiła istotna zmiana stosu technologicznego po stronie backendu – początkowo planowany Spring Boot (Java) został zastąpiony przez **Go** z routerem **chi** oraz generatorem kodu **sqlc**. Decyzja była motywowana chęcią wypróbowania języka Go w praktyce.

2 Stan implementacji

2.1 Struktura repozytorium

Projekt zorganizowany jest jako monorepo zawierające dwie aplikacje:

- `apps/task-api` – backend w Go (REST API),
- `apps/task-frontend` – frontend w React 19 (SPA),

oraz konfigurację infrastruktury (`docker-compose.yml`, `docker/keycloak/realm.json`, `flake.nix`).

2.2 Funkcjonalności zaimplementowane

- **Uwierzytelnianie i autoryzacja** – middleware HTTP weryfikujący tokeny JWT przy użyciu kluczy publicznych pobieranych z serwisu Keycloak (JWKS). Pierwsze żądanie nowego użytkownika powoduje automatyczną synchronizację profilu (utworzenie wpisu w tabeli `users` na podstawie pól `sub`, `name`, `email` z tokenu).
- **Zarządzanie zadaniami (CRUD)** – pełen zestaw operacji POST, GET, PUT, DELETE na zasobie zadań, zagnieżdżonym pod identyfikatorem użytkownika.
- **Filtrowanie listy zadań** – parametry zapytania `status` oraz `priority` są walidowane i przekazywane do dedykowanych zapytań SQL.
- **Dashboard** – agregujący endpoint zliczający zadania zgrupowane po statusie (`todo`, `in_progress`, `done`) oraz całkowitą liczbę zadań.
- **Walidacja danych wejściowych** – DTO posiadają metody `Validate()` sprawdzające obecność tytułu, dopuszczalne wartości statusu i priorytetu oraz format daty (YYYY-MM-DD).

- **Dokumentacja API** – automatycznie generowana specyfikacja OpenAPI / Swagger UI dostępna pod ścieżką `/swagger`, z konfiguracją OAuth2/PKCE wskazującą na realm Keycloak.
- **Health check** – endpoint `/health` weryfikujący dostępność bazy danych.
- **Frontend** – responsywny interfejs zbudowany w React 19, z Redux Toolkit (`createEntityAdapter`, `thunk`), komponentami z biblioteki `shadcn/ui` i stylowaniem Tailwind CSS v4. Strona `TasksPage` prezentuje statystyki, listę zadań oraz dialog tworzenia.
- **Konteneryzacja** – `docker-compose.yml` uruchamia PostgreSQL (osobne instancje dla aplikacji i Keycloak), Keycloak (z importem realmu) oraz panel Adminer.

2.3 Lista endpointów REST

Wszystkie endpointy są zabezpieczone middleware `Protect` (wymagany nagłówek `Authorization: Bearer <JWT>`) i udostępnione pod prefiksem `/api` (konfigurowalnym przez zmienną `API_BASE_PATH`).

Metoda	Ścieżka	Opis	Kody
GET	<code>/health</code>	Stan aplikacji i bazy danych	200, 401, 500
GET	<code>/users/{userId}/tasks</code>	Lista zadań użytkownika z opcjonalnymi filtrami	200, 400, 401, 500
POST	<code>/users/{userId}/tasks</code>	Utworzenie nowego zadania	201, 400, 401, 500
GET	<code>/users/{userId}/tasks/{id}</code>	Szczegóły pojedynczego zadania	200, 400, 401, 404, 500
PUT	<code>/users/{userId}/tasks/{id}</code>	Aktualizacja zadania	200, 400, 401, 404, 500
DELETE	<code>/users/{userId}/tasks/{id}</code>	Usunięcie zadania	204, 400, 401, 404, 500
GET	<code>/users/{userId}/dashboard</code>	Statystyki zadań użytkownika	200, 400, 401, 500

2.4 Schemat bazy danych

Migracje (`goose`) tworzą tabele `users` oraz `tasks`, a także typy wyliczeniowe `task_status` (`todo`, `in_progress`, `done`) i `task_priority` (`low`, `medium`, `high`). Tabela `tasks` posiada klucz obcy `user_id` z kaskadowym usuwaniem oraz znaczniki czasowe `created_at`, `updated_at`.

3 Stos technologiczny – aktualizacja

- **Frontend:** React 19, TypeScript, Vite 8, Tailwind CSS v4, `shadcn/ui`, React Router v7, Redux Toolkit, `react-redux`.

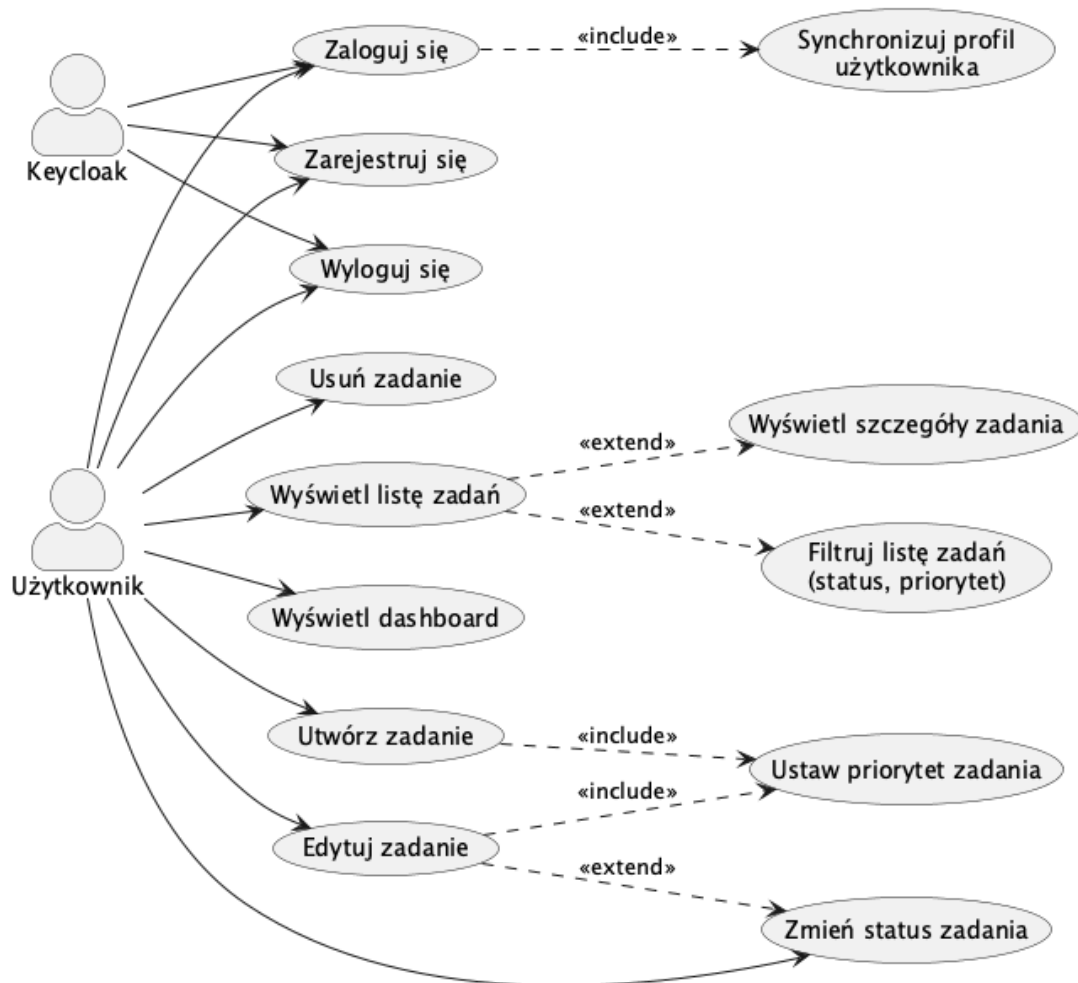
- **Backend:** Go (chi router, sqlc, goose, slog, swaggo).
- **Baza danych:** PostgreSQL 16.
- **Uwierzytelnianie:** Keycloak 26.2 (OAuth2 / OpenID Connect, PKCE, JWKS).
- **Konteneryzacja:** Docker, Docker Compose.
- **Środowisko deweloperskie:** Nix flake (flake.nix) oraz Taskfile.yml.
- **Kontrola wersji:** Git.

4 Zaktualizowane diagramy z kamienia I

Wszystkie diagramy zostały dostosowane do rzeczywistej implementacji.

4.1 Diagram przypadków użycia

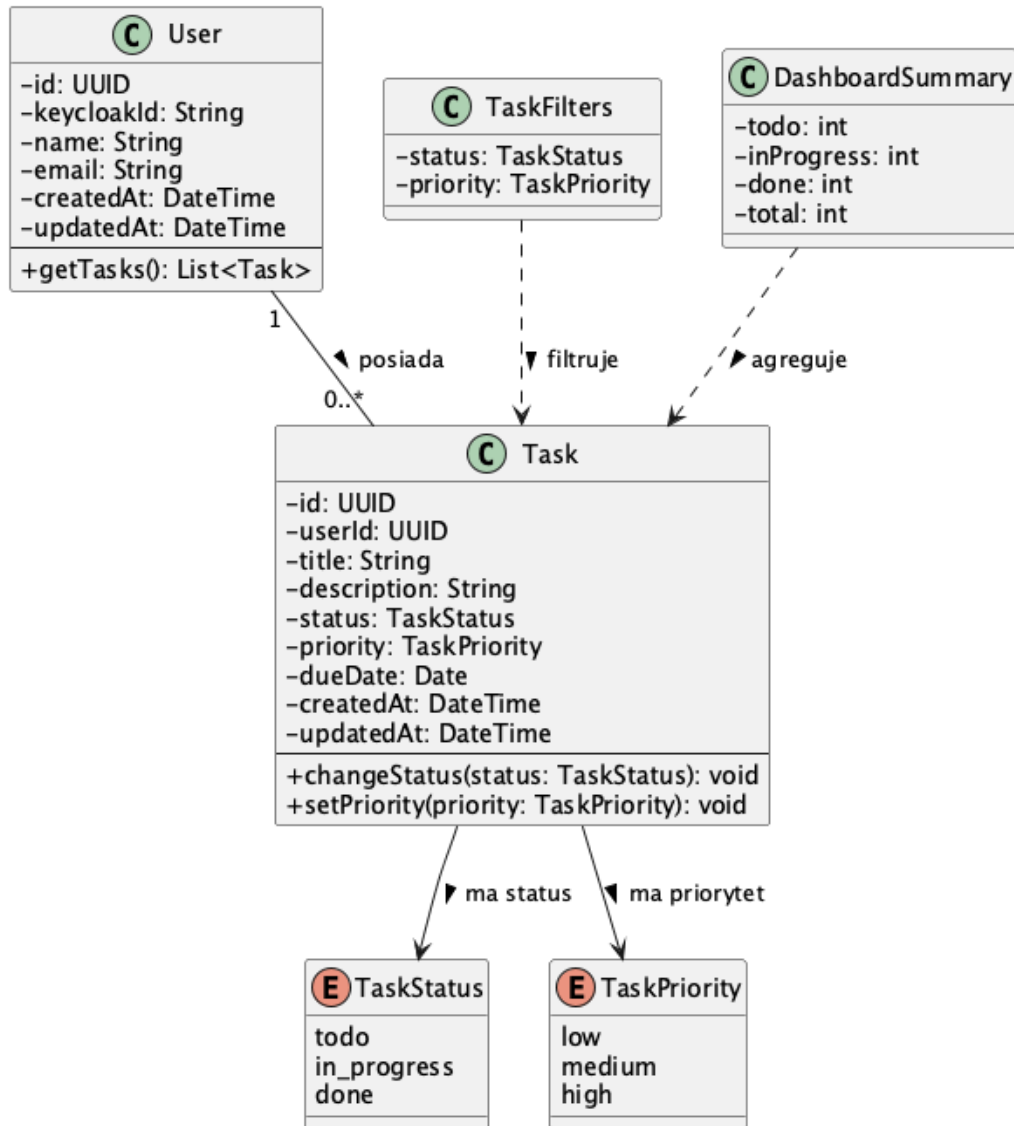
W stosunku do wersji z kamienia I dodany został przypadek *Synchronizuj profil użytkownika*, włączany przez przypadek *Zaloguj się* – odpowiada on faktycznemu mechanizmowi tworzenia rekordu w tabeli *users* przy pierwszym żądaniu autoryzowanym.



Rysunek 1: Diagram przypadków użycia

4.2 Diagram klas

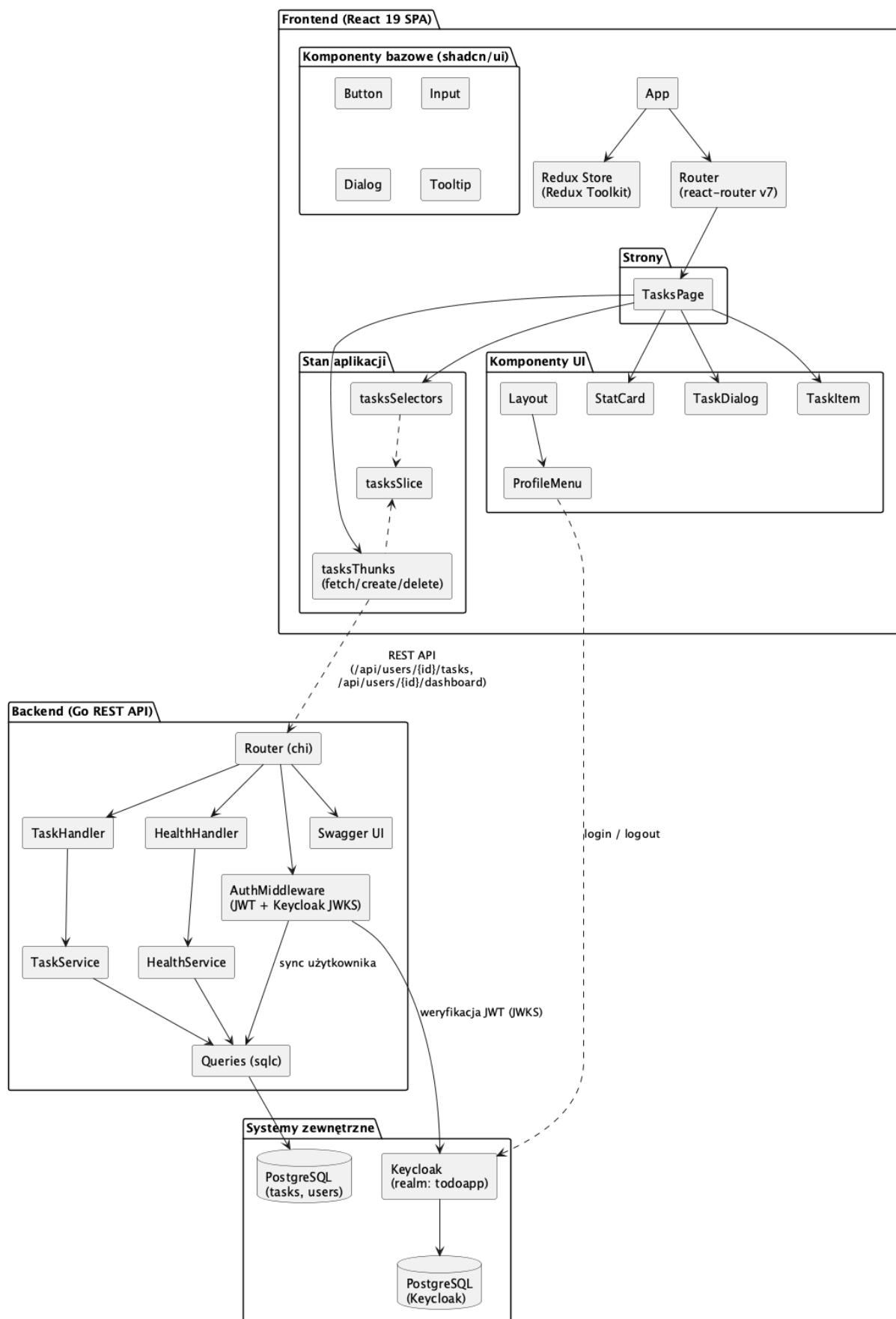
Klasa `User` została rozszerzona o pola `name`, `email`, `createdAt`, `updatedAt`, zgodnie z migracją `00002_add_user_profile_columns.sql`. Klasa `Task` otrzymała pole `userId` (klucz obcy). Wartości typów wyliczeniowych dostosowano do nazw rzeczywistych (`todo`, `in_progress`, `done`; `low`, `medium`, `high`). Dodano klasy pomocnicze `TaskFilters` oraz `DashboardSummary` odpowiadające DTO obecnym w kodzie.



Rysunek 2: Diagram klas (model dziedziny + DTO)

4.3 Diagram komponentów

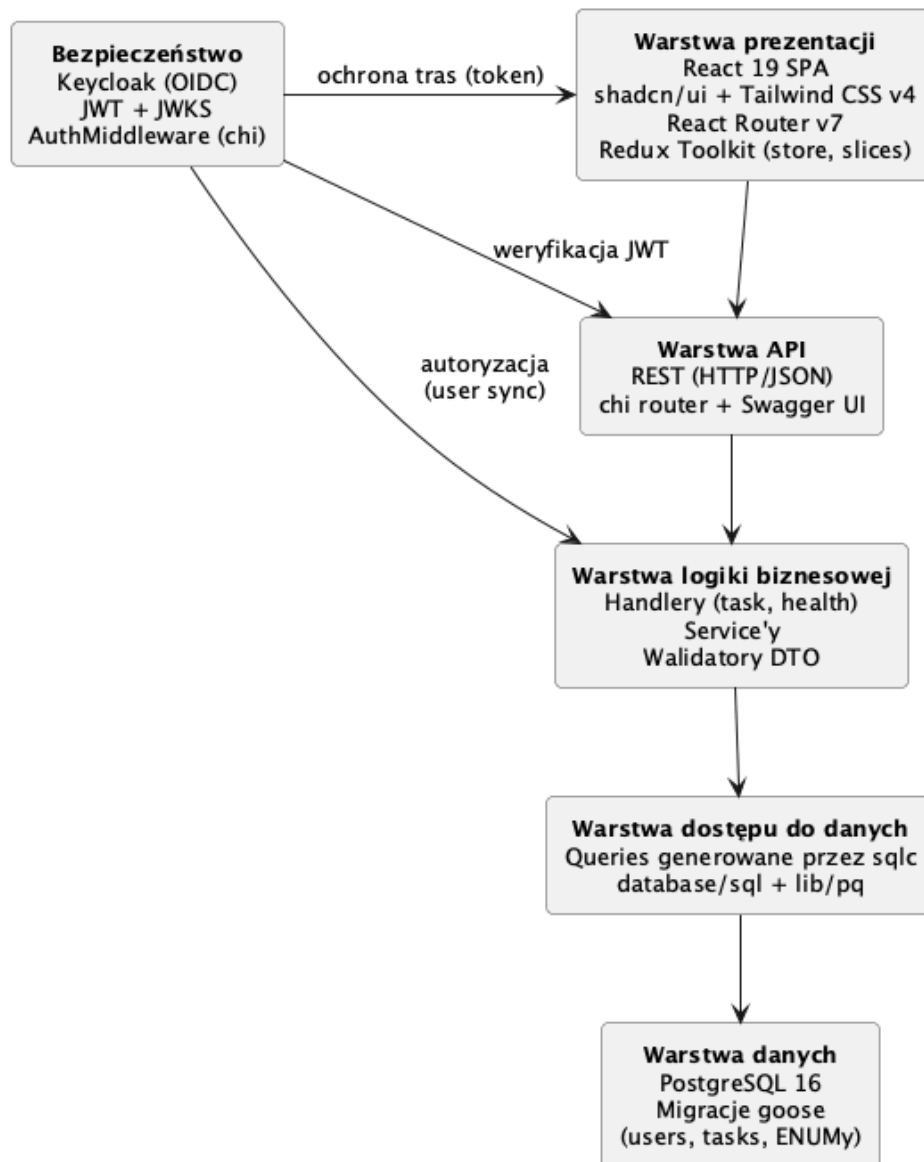
Diagram odzwierciedla rzeczywiste pakiety: po stronie frontendu wyodrębniono Redux Store (slice, thunki, selektory) oraz konkretne komponenty UI (`Layout`, `TaskDialog`, `TaskItem`, `StatCard`, `ProfileMenu`). Po stronie backendu przedstawiono router `chi`, `AuthMiddleware` z weryfikacją JWKS, handlery `TaskHandler` i `HealthHandler`, warstwę serwisów oraz wygenerowane przez `sqlc` Queries. Endpoint Swagger UI został wskazany jako osobny komponent.



Rysunek 3: Diagram komponentów

4.4 Diagram warstw systemu

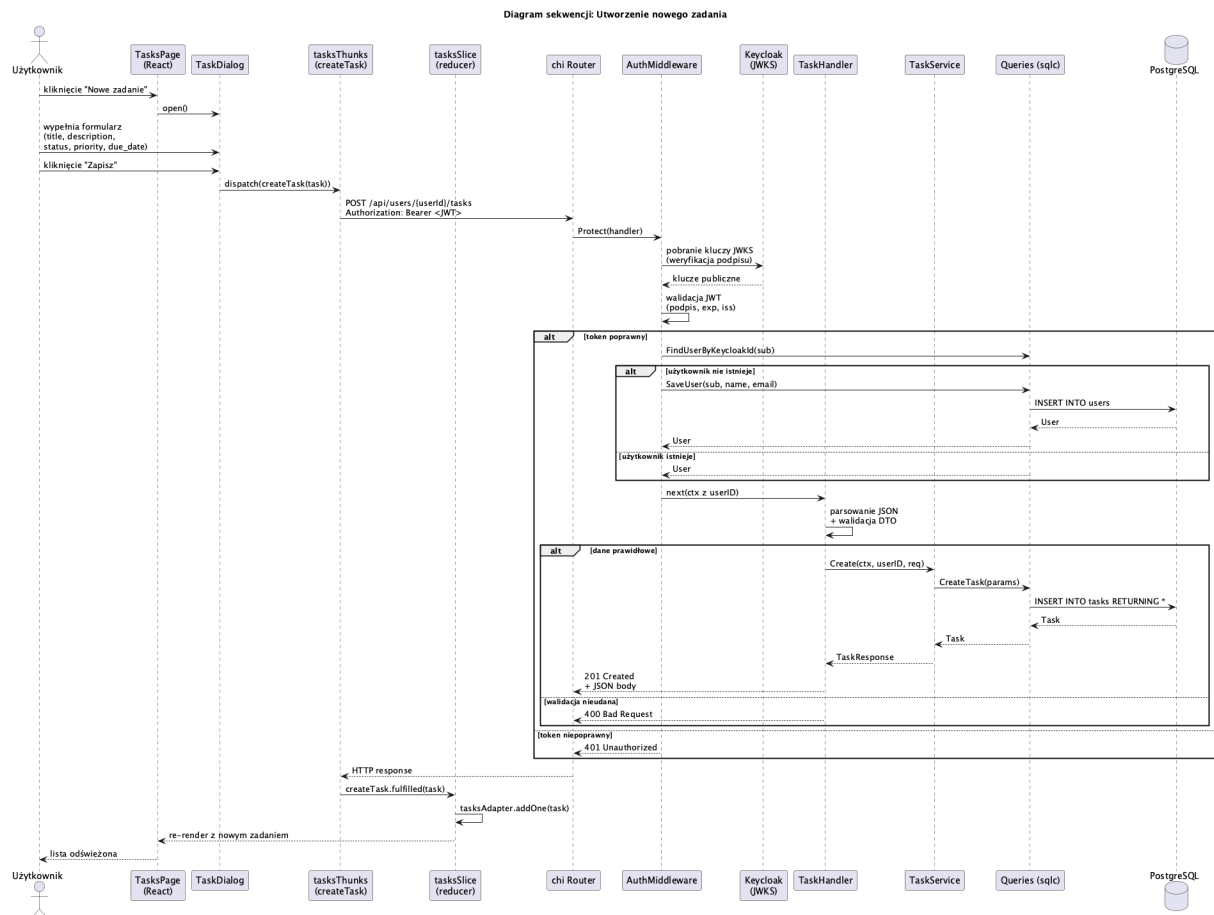
Etykiety warstw zaktualizowano zgodnie z zaimplementowanymi technologiami: warstwa prezentacji opisuje stack React 19 + Redux Toolkit, warstwa API – chi i Swagger UI, warstwa logiki – handlery i serwisy, warstwa dostępu do danych – generowane przez `sqlc` zapytania, a warstwa bezpieczeństwa – middleware weryfikujący JWT.



Rysunek 4: Diagram warstw systemu

5 Diagram sekwencji – utworzenie zadania

Wybrany scenariusz: użytkownik tworzy nowe zadanie. Scenariusz został wybrany, ponieważ ilustruje pełny przepływ przez wszystkie warstwy systemu – od interakcji w UI, przez akcję Redux i żądanie HTTP, weryfikację tokenu, synchronizację użytkownika, walidację DTO, persystencję w bazie, aż po aktualizację stanu po stronie klienta.



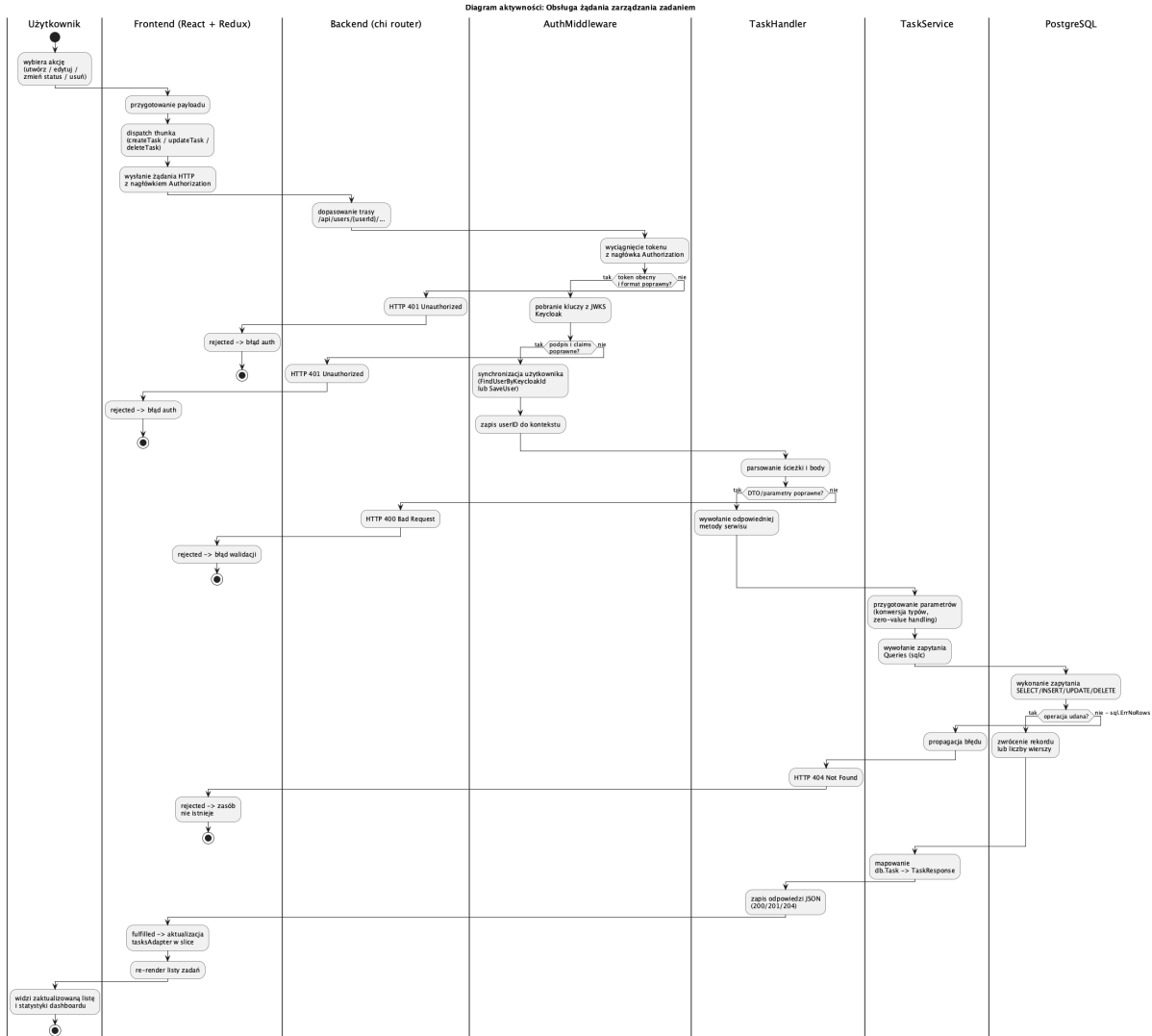
Rysunek 5: Diagram sekwencji: utworzenie nowego zadania

Opis kluczowych kroków.

1. Użytkownik otwiera dialog **TaskDialog** ze strony **TasksPage** i zatwierdza formularz.
2. Frontend wywołuje thunk **createTask**, który wysyła żądanie **POST /api/users/{userId}/tasks** wraz z tokenem JWT.
3. **AuthMiddleware** pobiera klucze z JWKS Keycloak i weryfikuje podpis tokenu.
4. Jeżeli użytkownik o podanym **sub** nie istnieje w bazie, jest tworzony (synchronizacja profilu); w przeciwnym wypadku rekord jest tylko odczytywany.
5. **TaskHandler** parsuje ciało żądania, uruchamia **Validate()** na DTO, a następnie wywołuje **TaskService.Create**.
6. Serwis przekazuje parametry do wygenerowanego zapytania **CreateTask**, które wykonuje **INSERT INTO tasks RETURNING ***.
7. Odpowiedź **201 Created** z zserializowanym zasobem trafia do thunka, który wywołuje akcję **createTask.fulfilled** – reducer dodaje encję do **tasksAdapter**, co skutkuje ponownym renderowaniem listy.

6 Diagram aktywności – zarządzanie zadaniem

Wybrany proces: ogólna obsługa żądania zarządzania zadaniem (utworzenie, edycja, zmiana statusu, usunięcie). Diagram zawiera punkty decyzyjne dla wszystkich istotnych ścieżek błędów obecnych w implementacji: 401 (brak/niepoprawny token), 400 (błędne dane wejściowe) oraz 404 (`sql.ErrNoRows`).



Rysunek 6: Diagram aktywności: obsługa żądania zarządzania zadaniem

Tory (swimlanes). Diagram wyodrębnia odpowiedzialności komponentów: *Użytkownik*, *Frontend (React + Redux)*, *Backend (chi router)*, *AuthMiddleware*, *TaskHandler*, *TaskService* oraz *PostgreSQL*.

7 Uruchamianie i weryfikacja

7.1 Wymagania wstępne

Docker, Docker Compose, Go 1.22+, Node.js 20+ oraz **task** (Taskfile).

7.2 Kroki uruchomienia

1. Skopiowanie pliku `.env.example` do `.env` i uzupełnienie zmiennych.
2. Start usług infrastrukturalnych: `docker compose up -d postgres postgres-keycloak keycloak adminer`.
3. Wykonanie migracji bazy danych: `task migrate` (lub `goose`).
4. Uruchomienie API: `task api:dev` (`go run ./cmd`).
5. Uruchomienie frontendu: `npm run dev` w katalogu `apps/task-frontend`.

7.3 Weryfikacja

Po starcie systemu dostępne są:

- Swagger UI – `http://localhost:<API_PORT>/swagger/index.html`,
- panel Keycloak – `http://localhost:8080`,
- Adminer – `http://localhost:8081`,
- frontend – `http://localhost:5173`.

8 Podsumowanie

W ramach drugiego kamienia milowego dostarczono działającą wersję systemu obejmującą podstawowe scenariusze użycia: uwierzytelnianie przez Keycloak, zarządzanie zadaniami (CRUD), filtrowanie oraz dashboard. Dokumentacja projektowa została rozszerzona o dwa nowe diagramy (sekwencji oraz aktywności), a wcześniej przygotowane diagramy zaktualizowano zgodnie z rzeczywistą implementacją – w tym kluczową zmianą stosu backendowego (Go zamiast Spring Boot) oraz rozszerzonym modelem użytkownika.